

FDU 数据库 6. 事务管理

本文参考以下教材:

- Database System Concepts (A. Silberschatz, H. Korth, S. Sudarshan) Chapter 17, 18, 19
- 数据库系统概念 (A. Silberschatz, H. Korth, S. Sudarshan, 杨冬青等译) 第 17, 18, 19 章

欢迎批评指正!

6.1 事务

6.1.1 定义

事务 (transaction) 是访问并可能更新各种数据项的一个程序执行单元, 由 `begin transaction` 与 `end transaction` 之间所执行的全部操作组成.

- 事务是不可分割的, 要么执行其全部操作, 要么就根本不执行.
因此如果一个事务开始执行, 但由于某种原因出错了 (例如发生零除、操作系统崩溃等), 那么事务对数据库所做的任何可能的修改都必须被撤销.
这种 "全或无" 的特性被称为**原子性** (atomicity).
- 事务必须保持数据库的一致性.
如果事务从一个一致的数据库开始以原子方式隔离地运行, 那么该数据库在事务结束时必须重新保持一致.
这种特性被称为**一致性** (consistency).
- 事务是一个单一的单元, 它的操作不能被并发执行的数据库操作所干扰.
这种特性被称为**隔离性** (isolation).
- 在一个事务成功完成之后, 它对数据库的改变必须是永久的, 即使出现系统故障也是如此.
这种特性被称为**持久性** (durability).

上述特性通常被称为 **ACID 特性**.

6.1.2 原子性和持久性

事务并非总能成功地执行, 此类事务将处于**中断** (aborted) 状态.
如果要确保原子性, 中断的事务对数据库所做过的任何修改都必须被撤销.
一旦中断的事务造成的修改已被撤销, 我们就称该事务**已回滚** (rolled back).

管理事务中断是恢复机制职责的一部分, 其经典实现是维护一个**日志** (log), 事务对数据库所做的所有修改都会先被记录到日志中.
对日志的维护提供了通过重做修改来保证原子性和持久性的可能, 以及在事务执行期间发生故障的情况下通过撤销修改来保证原子性的可能.

成功完成其执行的事务被称为**已提交** (committed) 的事务.
一旦事务已经提交, 我们就不能通过中断它来撤销其造成的影响.
撤销已提交事务所造成的影响的唯一方式是执行一个**补偿事务** (compensating transaction).
例如, 如果一个事务给一个账户增加了 20 人民币, 则其补偿事务应当从该账户减去 20 人民币.
但我们并非总是能够创建这样的补偿事务.
因此编写和执行补偿事务的责任被留给了用户, 而不是通过数据库系统来处理.

一个事务只能处于以下状态之一:

- **活跃** (active): 事务正在被执行.

- **部分提交** (partially committed): 事务的最后一条语句已被执行 (但实际输出可能仍临时贮存在主存中)
- **失败** (failed): 正常执行不能再继续.
- **中断** (aborted): 事务已回滚, 数据库已被恢复到它在事务开始前的状态.
- **提交** (committed): 事务已完成执行.

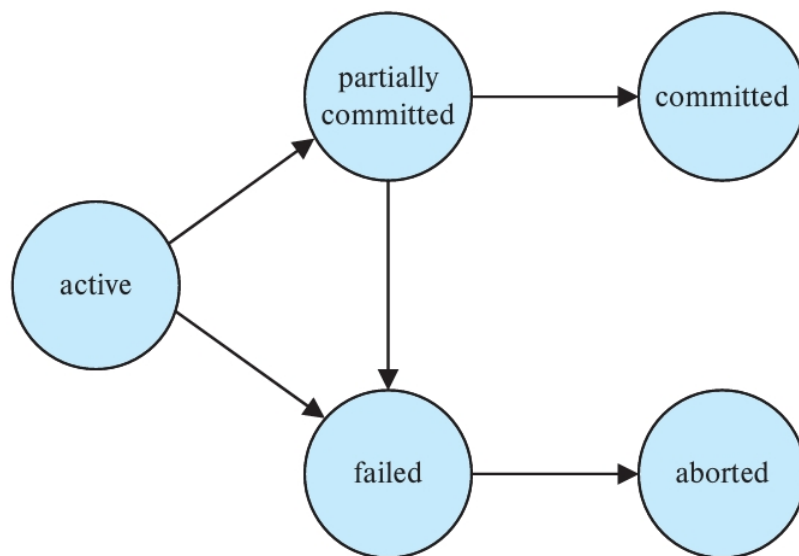


Figure 17.1 State diagram of a transaction.

6.1.3 隔离性

事务处理系统通常允许多个事务并发地执行.

数据库系统必须控制并发事务之间的交互, 以防止它们破坏数据库的一致性.

指令在系统中执行的时间顺序称为**调度** (schedule).

如果并发执行的控制完全由操作系统负责, 那么许多调度都是可能的:

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.2 Schedule 1—a serial schedule in which T_1 is followed by T_2 .

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.3 Schedule 2—a serial schedule in which T_2 is followed by T_1 .

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

在并发执行的情况下，我们需要保证所执行的任何调度的效果都与串行调度的效果一样。换言之，调度应该在某种意义上等价于一个串行调度。这种调度称为**可串行化的** (serializable) 调度。

如果调度 S_1 可以经过一系列非冲突指令的交换而转换成调度 S_2 ，则称 S_1 与 S_2 是**冲突等价的** (conflict equivalent)。若一个调度 S 与一个串行调度是冲突等价的，则称调度 S 是**冲突可串行化的** (conflict serializability)。

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Figure 17.6 Schedule 3—showing only the read and write instructions.

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

Figure 17.8 Schedule 6—a serial schedule that is equivalent to schedule 3.

为确定调度 S 是否是冲突可串行化的，我们可构造一个有向图，称为**优先图** (precedence graph). 其顶点集由参与到调度中的所有事务组成，边集由满足以下两个条件之一的所有 $T_i \rightarrow T_j$ 的边组成:

- ① 在 T_i 执行 write(Q) 之后, T_j 执行 read(Q) 或 write(Q)
- ② 在 T_i 执行 read(Q) 之后, T_j 执行 write(Q)

如果在优先图中存在一条 $T_i \rightarrow T_j$ 的边, 则在等价于 S 的任何串行调度 S' 中, T_i 必须出现在 T_j 之前. 如果调度 S 的优先图中无环, 则调度 S 是冲突可串行化的; 否则调度 S 是非冲突可串行化的.

为测试冲突可串行化性, 我们需要构造优先图并调用一个环路检测算法. 基于深度优先搜索的环路检测算法需要 n^2 量级的运算, 其中 n 是图中的顶点数 (即事务数).

T_1	T_2
$\text{read}(A)$ $A := A - 50$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$
$\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$B := B + \text{temp}$ $\text{write}(B)$ commit

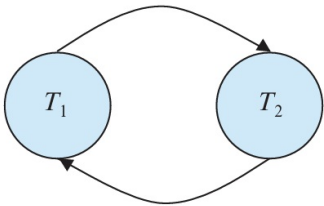


Figure 17.11 Precedence graph for schedule 4.

Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

通过寻找与优先图的偏序相一致的线性次序可以得到事务的**可串行化次序** (serializability order). 该过程称为**拓扑排序** (topological sorting). 一般而言, 通过拓扑排序可以得到几种可能的线性次序.

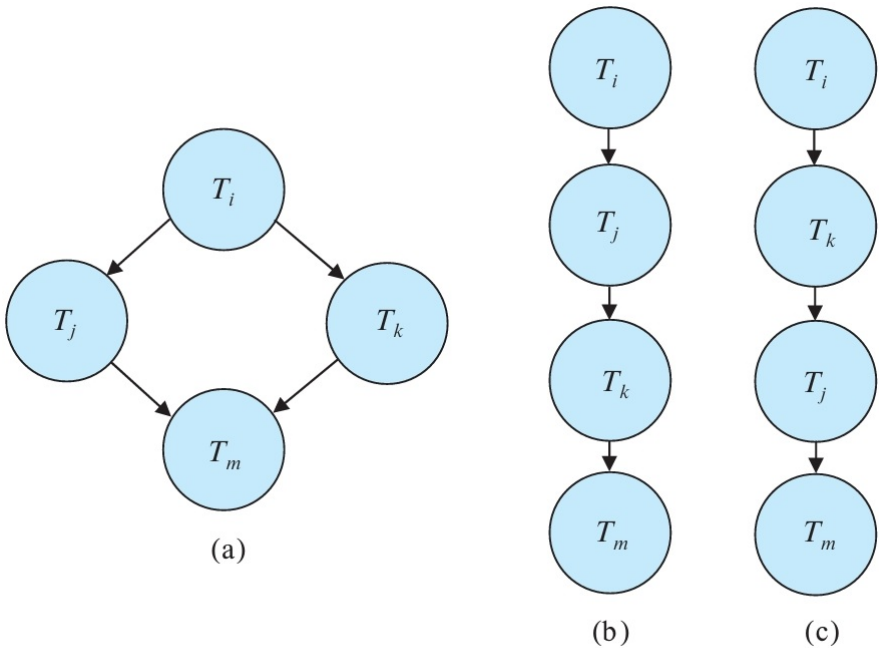


Figure 17.12 Illustration of topological sorting.

有可能存在两个调度, 它们产生相同的结果, 但它们不是冲突等价的. 例如下图中的调度的优先图有环, 因此不是冲突可串行化的, 但它产生的结果与串行执行 T_1, T_5 的结果相同. 因此冲突等价要比结果等价要更加严格.

T_1	T_5
read(A) $A := A - 50$ write(A)	
	read(B) $B := B - 10$ write(B)
read(B) $B := B + 50$ write(B)	
	read(A) $A := A + 10$ write(A)

Figure 17.13 Schedule 8.

现假设并发执行过程中事务有可能失败.

- ① 可恢复的 (reconverable) 调度:

对于每对事务 T_i 和 T_j , 如果 T_j 读取了由 T_i 在之前写过的数据项, 则 T_j 的提交操作应安排在 T_i 的提交操作之后.

下图展示了一个不可恢复的调度:

T_6	T_7
read(A) write(A)	
	read(A) commit
read(B)	

Figure 17.14 Schedule 9, a nonrecoverable schedule.

- ② 无级联调度:

因单个事务失败而导致一系列事务回滚的现象称为级联回滚 (cascading rollback).

而无级联调度是这样的一种调度:

对于每对事务 T_i 和 T_j , 如果 T_j 要读取由 T_i 在之前写过的数据项, 则该读取操作必须在 T_i 提交后进行.

显然, 无级联调度一定是可恢复调度.

6.2 并发控制

6.2.1 基于锁的协议

确保隔离性的一种方式是对数据项以互斥的方式进行访问。

也就是说，当一个事务访问某个数据项时，其他任何事务都不能修改该数据项。

最常用的实现方法是：

当且仅当一个事务当前持有一个数据项上的**锁** (lock) 的情况下，该事务才能够访问这个数据项。

简单起见，我们只考虑两种加锁方式：

- ① **共享锁**：

如果一个事务 T 获得了数据项 Q 上的共享锁 S ，则 T 可以读 Q ，但不能写 Q 。

- ② **排他锁**：

如果一个事务 T 获得了数据项 Q 上的排他锁 X ，则 T 既可以读 Q 又可以写 Q 。

在任何时候，一个特定数据项上可同时 (被不同事务) 持有多个共享锁。

而排他锁必须等到对应数据项上的共享锁或排他锁被释放为止。

以下调度显示了由事务执行的操作以及并发控制管理器授权锁的时刻：

	T_1	T_2	concurrency-control manager
T_1 : lock-X(B); read(B); $B := B - 50$; write(B); unlock(B); lock-X(A); read(A); $A := A + 50$; write(A); unlock(A).	lock-X(B) read(B) $B := B - 50$ write(B) unlock(B)	 lock-S(A) read(A) unlock(A) lock-S(B) read(B) unlock(B) display($A + B$)	grant-X(B, T_1) grant-S(A, T_2) grant-S(B, T_2) grant-X(A, T_1)
T_2 : lock-S(A); read(A); unlock(A); lock-S(B); read(B); unlock(B); display($A + B$).	lock-X(A) read(A) $A := A + 50$ write(A) unlock(A)		

Figure 18.2 Transaction T_1 .

Figure 18.3 Transaction T_2 .

Figure 18.4 Schedule 1.

然而加锁机制可能会造成**死锁** (deadlock):

不同事务各自持有一些锁，同时等待对方释放另一些锁，导致都无法正常执行。

T_3	T_4
lock-X(B) read(B) $B := B - 50$ write(B)	
	lock-S(A) read(A) lock-S(B)
lock-X(A)	

Figure 18.7 Schedule 2.

当死锁发生时，系统必须回滚这两个事务中的一个。
一旦一个事务被回滚，被该事务锁住的数据项就被解锁。
另一个事务就可以访问这些数据项，从而继续它的执行。

保证可串行化的一种协议是**两阶段加锁协议** (two-phase locking protocol):

- ① 增长阶段 (growing phase): 一个事务可以获得锁，但不能释放任何锁。
- ② 缩减阶段 (shrinking phase): 一个事务可以释放锁，但不能获得任何新锁。

两阶段封锁并不保证不会发生死锁:

T_3	T_4
lock-X(B) read(B) $B := B - 50$ write(B)	
	lock-S(A) read(A) lock-S(B)
lock-X(A)	

Figure 18.7 Schedule 2.

除了希望调度可串行化之外，调度还应该无级联的。
在两阶段封锁协议下，级联回滚是可能发生的。
考虑下图中的调度，每个事务都遵循两阶段封锁协议，
但如果在 T_7 的 read(A) 步骤之后 T_5 发生故障，就会导致 T_5, T_6, T_7 的级联回滚。

T_5	T_6	T_7
lock-X(A) read(A) lock-S(B) read(B) write(A) unlock(A)	lock-X(A) read(A) write(A) unlock(A)	lock-S(A) read(A)

Figure 18.8 Partial schedule under two-phase locking.

级联回滚可以通过将两阶段加锁协议修改为严格两阶段加锁协议来避免。这种协议不但要求封锁是两阶段的，而且还要求事务所持有的所有排他锁必须在事务提交后方可释放。这个要求保证未提交事务所写的任何数据在该事务提交之前均以排他锁保护，从而防止其他任何事务读取这些数据。

请考察下面两个事务：

T_8 : read(a_1);
read(a_2);
...
read(a_n);
write(a_1).

T_9 : read(a_1);
read(a_2);
display($a_1 + a_2$).

如果我们采用两阶段封锁协议，则必须以排他锁包含 a_1 ，这会使两个事务的任何并发执行都相当于一种串行执行。但 T_8 仅在其执行末尾才需要 a_1 上的排他锁。因此如果 T_8 开始时以共享模式封锁 a_1 ，随后将该锁变更为排他锁的话，那么我们可以获得更高的并发度，因为 T_8 与 T_9 可以同时访问 a_1 与 a_2 。

上述观察指导我们对基本的两阶段封锁协议加以改进，使之允许锁转换 (lock conversion)。我们通过**升级** (upgrade) 来将共享锁变更为排他锁，通过**降级** (downgrade) 来将排他锁变更为共享锁。锁转换并不允许随意进行：升级只能发生在增长阶段，而降级只能发生在缩减阶段。

树型协议 (tree protocol)

假设所有数据项的集合 D 上存在一种偏序, 可将集合 D 组织成有向无环图, 称为**数据库图**.

只使用排他锁, 每个事务对一个数据项至多加一次锁, 且遵从以下规则:

- ① 事务 T 的首次加锁可在任意数据项上进行.
- ② 此后事务 T 可以对一个数据项 Q 加锁的前提是 T 持有 Q 的父项上的锁.
- ③ 对数据项解锁的操作可以随时进行.
- ④ 事务 T 对一个数据项加锁并解锁后, 不能再对该数据项加锁.

在树型协议下的所有合法调度是冲突可串行化的.

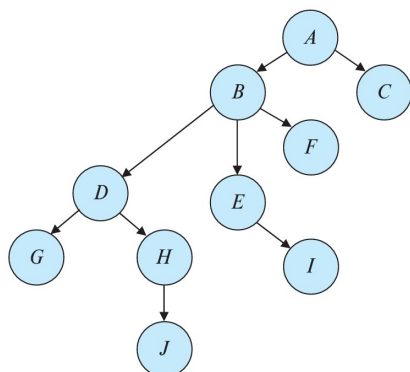


Figure 18.11 Tree-structured database graph.

T_{10} : lock-X(B); lock-X(E); lock-X(D); unlock(B); unlock(E); lock-X(G);
 unlock(D); unlock(G).
 T_{11} : lock-X(D); lock-X(H); unlock(D); unlock(H).
 T_{12} : lock-X(B); lock-X(E); unlock(E); unlock(B).
 T_{13} : lock-X(D); lock-X(H); unlock(D); unlock(H).

T_{10}	T_{11}	T_{12}	T_{13}
lock-X(B)	lock-X(D) lock-X(H) unlock(D)		
lock-X(E) lock-X(D) unlock(B) unlock(E)		lock-X(B) lock-X(E)	
lock-X(G) unlock(D)	unlock(H)		lock-X(D) lock-X(H) unlock(D) unlock(H)
unlock(G)		unlock(E) unlock(B)	

Figure 18.12 Serializable schedule under the tree protocol.

与两阶段封锁不同, 树型协议不会产生死锁, 因此无须回滚.

此外, 树型封锁协议的另一个优点是可以较早释放锁, 缩短等待时间, 增加并发度.

但树型协议也有缺点: 在某些情况下, 一个事务可能必须给它根本不访问的数据项加锁.

(时间戳排序协议)

对于系统中的事务 T , 我们记其时间戳为 $TS(T)$.

我们将每个数据项 Q 与两个时间戳值相关联:

- W-timestamp(Q) 代表成功执行 write(Q) 的所有事务中的最大时间戳
- R-timestamp(Q) 代表成功执行 read(Q) 的所有事务中的最大时间戳

每当有新的 read(Q) 或 write(Q) 指令被执行时, 上述时间戳就被更新.

时间戳排序协议保证任何有冲突的 read 和 write 操作按时间戳的次序执行:

- 假设事务 T 发出 read(Q):
 - 若 $TS(T) < W\text{-timestamp}(Q)$, 则 T 需要读取的 Q 值已被覆盖, read(Q) 操作被拒绝, 事务 T 被回滚.
 - 若 $TS(T) \geq W\text{-timestamp}(Q)$, 则执行 read(Q) 操作, 并置 R-timestamp(Q) 为 $\max(R\text{-timestamp}(Q), TS(T))$
- 假设事务 T 发出 write(Q):
 - 若 $TS(T) < W\text{-timestamp}(Q)$ 或 $TS(T) < R\text{-timestamp}(Q)$, 则 T 产生的 Q 值已过时或已不再被需要, write(Q) 操作被拒绝, 事务 T 被回滚.
 - 否则执行 write(Q) 操作, 并置 W-timestamp(Q) 为 $TS(T)$

6.2.2 死锁处理

如果系统中的每个事务都在等待某一个事务释放锁，那么我们称该系统处于死锁状态。

预防死锁有三种方法。

- ① 要求同时获得所有的锁来保证不会发生循环等待。
但在事务开始之前，我们通常很难预知哪些数据项需要加锁，同时这种粗粒度的加锁会降低并发度。
- ② 按照某种预定的次序来加锁，类似于树型协议。
- ③ 每当等待有可能导致死锁时，它就执行事务回滚而不是等待锁。
一种简单的实现是**锁超时** (lock timeout): 申请锁的事务至多等待一段指定的时间。
若在那段时间内锁尚未授予给该事务，则称该事务超时，并自动回滚并重启。
如果确实存在死锁，则某些事务将超时并回滚，使得其他事务得以继续执行。

还有两种基于时间戳的死锁预防机制：

- **等待-死亡 (wait-die) 机制**是一种非抢占技术。
当事务 T_1 申请的数据项被 T_2 持有时，当且仅当 T_1 的时间戳小于 T_2 的时间戳时允许 T_1 等待。
否则 T_1 回滚 (死亡)。
- **伤害-等待 (wound-wait) 机制**是一种抢占技术。
当事务 T_1 申请的数据项被 T_2 持有时，当且仅当 T_1 的时间戳大于 T_2 的时间戳时允许 T_1 等待。
否则 T_2 回滚 (被 T_1 伤害)。

这两种机制都可能造成不必要的回滚。

死锁可以用一种称为**等待图** (wait-for graph) 的有向图来描述。

当事务 T_i 申请的数据项当前被事务 T_j 持有时，边 $T_i \rightarrow T_j$ 被插入等待图中。

只有当事务 T_j 不再持有事务 T_i 所需的数据项时，这条边才被删除。

当且仅当等待图中包含环路时，系统中存在死锁。

陷入该环路中的每个事务被称为处于死锁状态。

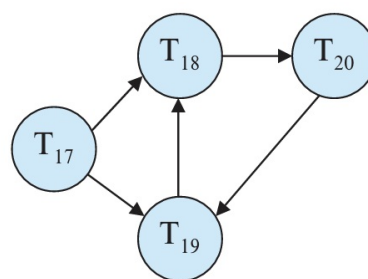
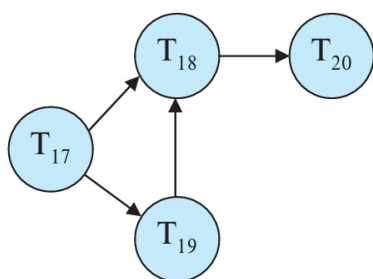


Figure 18.13 Wait-for graph with no cycle. Figure 18.14 Wait-for graph with a cycle.

解除死锁最常用的解决方案是回滚某些事务：

- ① 选择牺牲者：根据回滚代价来选择应该回滚哪些事务。
- ② 决定要回滚到什么程度 (完全回滚当然是最简单粗暴的做法)
- ③ 饥饿问题：我们必须保证一个事务被选为牺牲者的次数是有限的。

6.2.3 多级粒度

目前为止，我们考虑的并发控制机制都将每个单独的数据项作为进行同步执行的单元。我们可以引入**多级粒度** (multiple granularity) 的机制，即定义一种数据粒度的层次结构，其中小粒度数据项嵌套在大粒度数据项中。

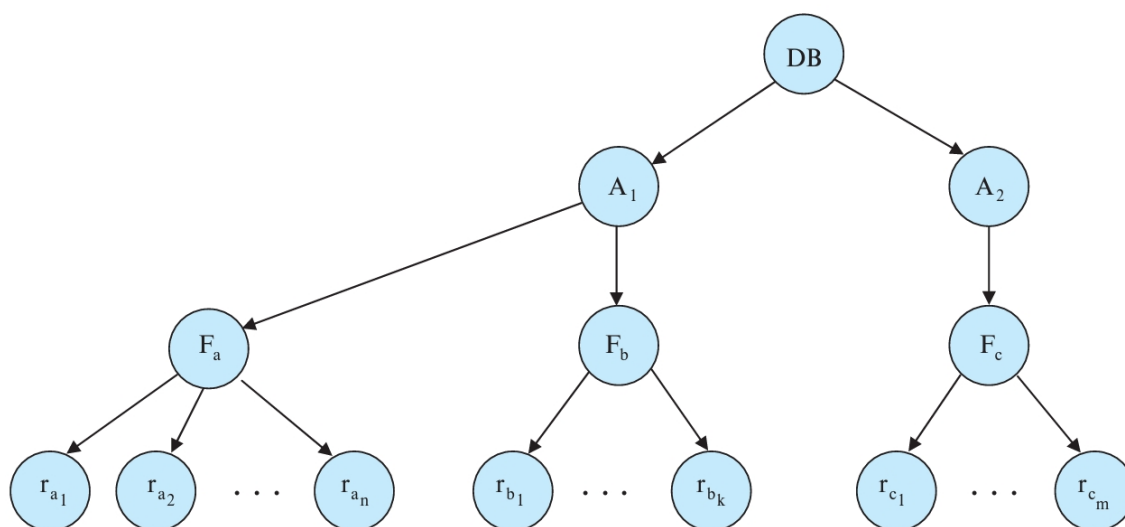


Figure 18.15 Granularity hierarchy.

值得注意的是，多粒度树中的非叶节点代表与其子节点存在包含关系的数据项。在树型协议中，每个节点都是一个独立的数据项。

此外，我们引入**意向锁** (intention lock) 机制。

如果一个节点加上了意向锁，则意味着要在树的较低层进行显式加锁。

在一个节点被显式加锁之前，该节点的全部祖先节点均要加上意向锁。

这样使得一个事务不必搜索整棵树就能判定它能否成功地给一个节点加锁。

希望给节点 Q 加锁的事务必须遍历树中从根节点到 Q 的一条路径。

在遍历树的过程中，该事务以意向模式对各节点加锁。

- 如果一个节点被加上了意向共享锁 IS，那么将在树的较低层加共享锁。
- 如果一个节点被加上了意向排他锁 IX，那么将在树的较低层加排他锁或共享锁。
- 如果一个节点被加上了共享意向排他锁 SIX，那么以该节点为根的子树以共享模式被显式加锁，并且将在树的更低层显式地加排他锁。

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
SIX	✓	×	×	×	×
X	×	×	×	×	×

Figure 18.16 Compatibility matrix.

多粒度加锁协议:

- 事务 T 必须遵守锁相容性函数.
- 事务 T 必须首先对树的根节点加锁 (可以任意的锁模式)
- 当且仅当事务 T 当前对 Q 的父节点具有意向排他锁 IX 或意向共享锁 IS 时, T 才能对 Q 加共享锁 S 或意向共享锁 IS.
- 当且仅当事务 T 当前对 Q 的父节点具有意向排他锁 IX 或共享意向排他锁 SIX 时, T 才能对 Q 加排他锁 X、意向排他锁 IX 或共享意向排他锁 SIX.
- 当且仅当事务 T 未对任何节点解锁时, T 才能对一个节点加锁 (即隐含着两阶段加锁协议)
- 当且仅当事务 T 不持有 Q 的任何子节点的锁时, T 才可对节点 Q 解锁.

6.3 系统恢复

6.3.1 故障分类

简单起见, 我们只考虑以下类型的故障:

- ① **事务故障 (transaction failure)**
有两种错误可能造成事务执行失败:
 - **逻辑错误 (logical error):**
事务由于某些内部情况而无法继续其正常执行, 这样的内部情况诸如非法输入、找不到数据、溢出或超出资源限制.
 - **系统错误 (system error):**
系统进入一种不良状态 (如死锁), 其结果是事务无法继续其正常执行. 但该事务可以在之后的某个时间重新执行.
- ② **系统崩溃 (system crash)**
硬件故障, 或者是数据库软件或操作系统的漏洞, 导致易失性存储器内容的丢失, 并使得事务处理停止. 但非易失性存储器内容完好无损且没被破坏.
- ③ **磁盘故障 (disk failure)**
在数据传输操作中由于磁头损坏或故障造成磁盘块上的内容丢失.

恢复算法由两部分组成:

- 在正常事务处理中采取措施，保证存在足够的信息可用于故障恢复.
- 故障发生后采取措施，将数据库内容恢复到某个保证数据库一致性、事务原子性及持久性的状态.

6.3.2 存储和访问

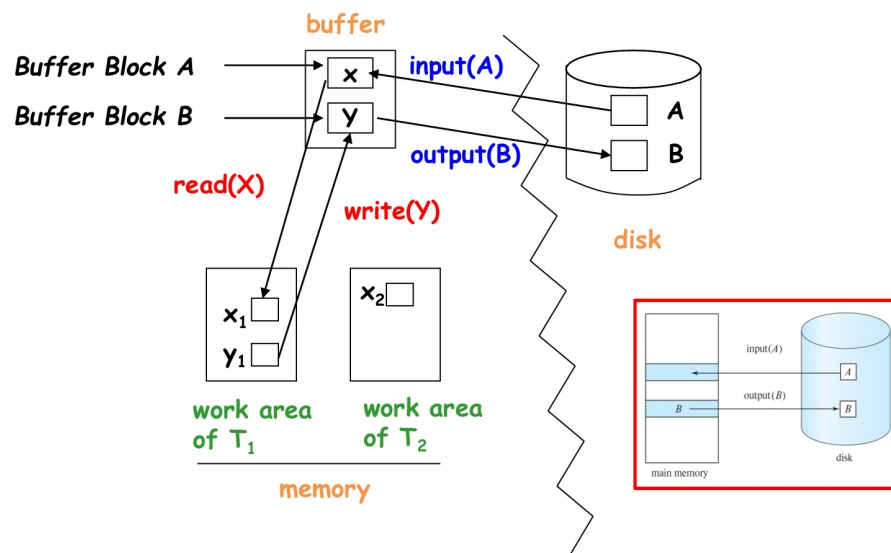
Storage Structure

- **Volatile storage (易失性存储器)**
 - does not survive system crashes
 - e.g., main memory, cache memory
- **Nonvolatile storage (非易失性存储器)**
 - survives system crashes
 - e.g., disk, tape, flash memory
- **Stable storage (稳定存储器)**
 - a mythical form of storage that survives all failures
 - approximated by maintaining multiple copies on distinct nonvolatile media

Data Access

- **Physical blocks** (物理块)
 - the blocks residing on the disk
- **Buffer blocks** (缓冲块)
 - the blocks residing temporarily in main memory
- Block movements between disk and main memory
 - **input(B)**: physical block $\rightarrow$ memory
 - **output(B)**: buffer block $\rightarrow$ disk
- Each transaction T_i has its **private work-area** (私有工作区)
 - T_i 's **local copy** of a data item X is called x_i
- Transaction transfers data items between **system buffer blocks** and **its private work-area** using
 - **read(X)**
 - **write(X)**
- Transactions
 - Perform **read(X)** while accessing X for the first time
 - All subsequent accesses are to the **local copy**
 - After last access, transaction executes **write(X)**
- **output(B_x)** does not need to immediately follow **write(X)**
 - System can perform the output operation when it deems fit

Example of Data Access



6.3.3 恢复与原子性

Recovery and Atomicity

- Modifying the database without ensuring that the transaction will commit may leave the database in an **inconsistent state**
 - Consider transaction T_i that transfers \$50 from account **A** to account **B**
 - Several output operations may be required for T_i to output **A** and **B**
 - A failure may occur after one of these modifications have been made but before all of them are made
- To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself
- Two approaches
 - log-based recovery (基于日志的恢复)
 - shadow-paging (影子页)

日志是**日志记录** (log record) 的序列, 它记录了数据库中的所有更新活动.

更新日志记录描述一次数据库写操作, 它具有以下字段:

- **事务标识**: 执行 write 操作的事务的唯一标识.
- **数据项标识**: 是所写数据项的唯一标识.
通常是数据项在磁盘上的位置, 包括该数据项所驻留的块的块标识以及块内偏移量.
- **旧值**: 数据项的写前值.
- **新值**: 数据项写后应取的值.

我们将一条更新日志记录表示为 $\langle T, Q, V_1, V_2 \rangle$

代表事务 T 对数据项 Q 执行写操作, 写之前 Q 的值是 V_1 , 写之后 Q 的值是 V_2 .

Log-based Recovery

- A log is kept on stable storage
 - The log is a sequence of log records
- When transaction T_i starts, it registers itself by writing a **$\langle T_i \text{ start} \rangle$** log record
 - Before T_i executes **write(X)**, a log record $\langle T_i, X, V_1, V_2 \rangle$ is written, where V_1 is the **old value** and V_2 is the **new value**
 - When T_i finishes its last statement, the log record **$\langle T_i, \text{commit} \rangle$** is written
- Two approaches using logs
 - Deferred database modification (延迟数据库修改)
 - Immediate database modification (即刻数据库修改)

如果一个事务执行了对磁盘缓冲块或磁盘本身的更新, 我们说这个事务修改了数据库; 而事务在主存中私有部分所进行的更新不算数据库修改.

如果一个事务直至提交时都没有修改数据库, 就说它采用了**延迟修改** (deferred-modification) 技术.

如果在事务仍然活跃时就发生数据库修改, 就说它采用了**立即修改** (immediate-modification) 技术.

延迟修改的开销是, 事务需要做更新过的所有数据项的本地拷贝;

并且如果一个事务读它更新过的数据项, 它必须从自己的本地拷贝中读.

发生系统崩溃之后，系统为保证原子性而查阅日志，
以确定哪些事务需要进行重做，哪些事务需要进行撤销。

- 如果日志中包括 $\langle T \text{ start} \rangle$ 记录，但不包含 $\langle T \text{ commit} \rangle$ 或 $\langle T \text{ abort} \rangle$ 记录，那么这样的事务 T 需要进行撤销。
- 如果日志中包括 $\langle T \text{ start} \rangle$ 记录以及 $\langle T \text{ commit} \rangle$ 或 $\langle T \text{ abort} \rangle$ 记录，那么这样的事务 T 需要进行重做。

考虑以下三种情况：

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

Figure 19.4 The same log, shown at three different times.

- 对于第一种情况，我们需要撤销事务 T_0 ，将 A, B 的值分别恢复为 1000 和 2000。
- 对于第二种情况，我们需要重做事务 T_0 ，将 A, B 的值分别修改为 950 和 2050；同时撤销事务 T_1 ，将 C 的值恢复为 700。
- 对于第三种情况，我们需要重做事务 T_0, T_1 ，将 A, B, C 的值分别修改为 950, 2050, 600。

(检查点)

当发生系统故障时，我们必须查阅日志，决定哪些事务需要重做，哪些事务需要撤销，原则上，我们需要搜索整个日志来确定上述信息。

这种方式有两个主要的困难：

- 搜索过程太耗时
- 根据我们的算法，大多数需要重做的事务已经将其更新写入数据库中。
尽管重做它们不会造成不良后果，但会使恢复过程变得更长。

为降低这种开销，我们引入**检查点** (checkpoint) 机制。

一种简单的实现是在检查点操作过程中不允许执行任何更新，将所有修改过的缓冲块都写入磁盘：

- 将当前位于主存的所有日志记录输出到稳定存储器。
- 将所有修改过的缓冲块输出到磁盘。
- 将一条形如 $\langle \text{checkpoint } L \rangle$ 的日志记录输出到稳定存储器，
其中 L 是执行检查点时正活跃的事务的列表。

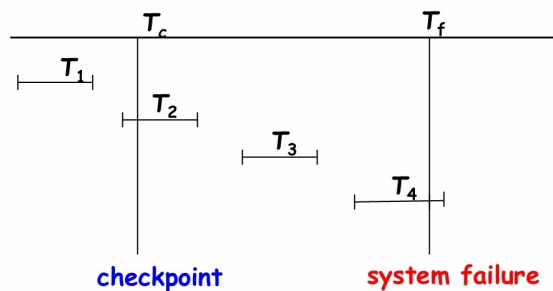
检查点执行过程中，不允许事务执行任何更新动作，如往缓冲块中写入或写日志记录。

当系统恢复时， $\langle \text{checkpoint } L \rangle$ 记录之前的所有数据库修改必然已经写入数据库，不必重做了，系统从日志末尾反向搜索，直至找到第一条 $\langle \text{checkpoint } L \rangle$ 记录未知。

Checkpoints (检查点)

- **Problems in the recovery procedure**
 - searching the entire log is time-consuming
 - might unnecessarily redo transactions which have already output their updates to the database
- **Recovery procedure by setting checkpoints periodically**
 - Output all log records currently residing in main memory to stable storage
 - Output all modified buffer blocks to the disk
 - Write a log record **<checkpoint>** to stable storage
- **During recovery**
 - Scan backwards from the end of log to find the most recent **<checkpoint>** record
 - Continue scanning backwards till a record **<T_i,start>** is found. We assume that all transactions are executed serially.
 - Need only consider the part of log following above start record
 - For all transactions (starting from **T_i** or later) with no **<T_i,commit>**, execute **undo(T_i)**.
 - Scanning forward in the log, for all transactions starting from **T_i** or later with a **<T_i,commit>**, execute **redo(T_i)**.

Example of Checkpoints



- T_1 can be ignored (updates already output to disk according to the checkpoint)
- T_2 and T_3 redone
- T_4 undone

6.3.4 恢复算法

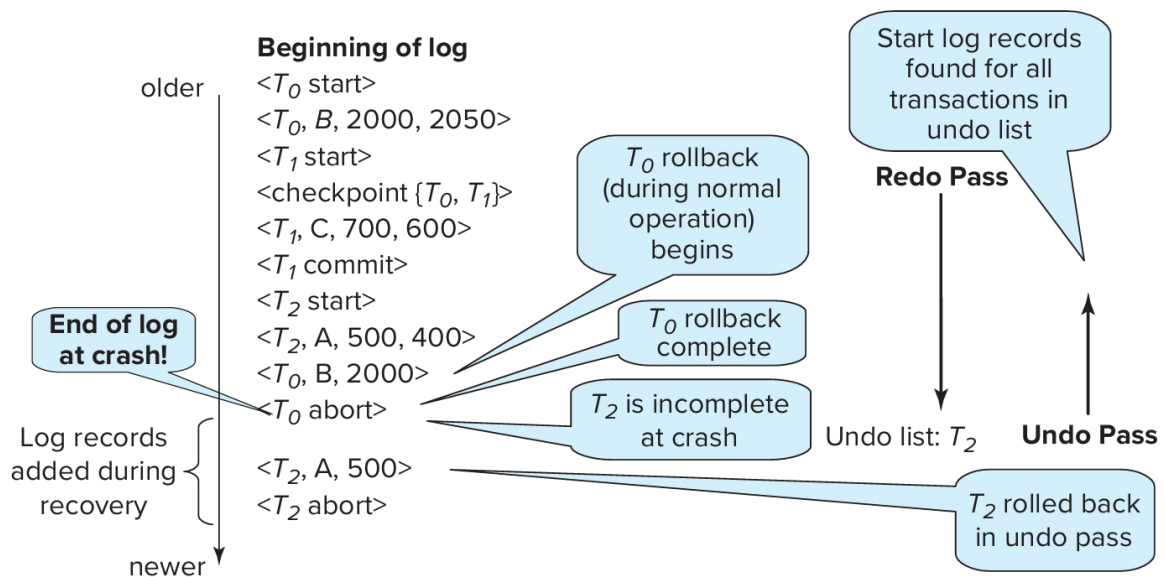


Figure 19.5 Example of logged actions and actions during recovery.

当从崩溃中恢复时，在重做阶段，系统对最后一个检查点记录之后的所有操作执行重做。

在这个阶段中，undo-list 列表中初始时包含 T_0 和 T_1 。

当 T_1 的 commit 日志记录被发现时，先从列表中去掉 T_1 ，

而当 T_2 的 start 日志记录被发现时， T_2 被加到列表中。

当事务 T_0 的 abort 日志记录被发现时， T_0 从 undo-list 中去掉，只剩 T_2 在 undo-list 中。

撤销阶段从尾端开始反向扫描日志，当发现 T_2 更新 A 的日志记录时，

就将 A 恢复成旧值，并往日志中写一条 redo-only 日志记录。

当发现 T_2 的 start 记录时，就为 T_2 添加一条 abort 记录。

由于 undo-list 中不再包含任何事务了，撤销阶段终止，恢复完成。

Recovery with Concurrent Transactions

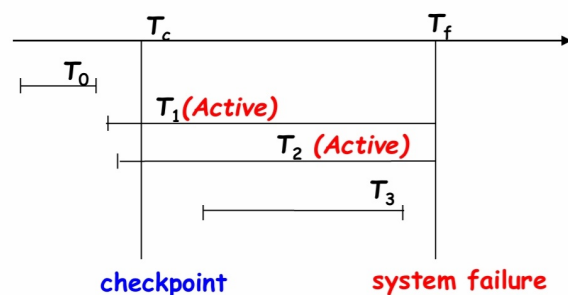
- We modify the **log-based recovery schemes** to allow **multiple transactions to execute concurrently**
 - All transactions **share a single disk buffer** and **a single log**
 - A **buffer block** can have data items updated by one or more transactions
- We **assume concurrency control using strict two-phase locking**
- Logging is done as described earlier
 - Log records of different transactions may be interspersed (散布) in the log
- The **checkpointing technique** and actions taken on recovery have to be changed
- Checkpoints are performed as before, except that the checkpoint log record is the form **<checkpoint L>**
 - L** is a list of transactions **active** at the time of the **checkpoint**
 - We assume no update is in progress while the checkpoint is carried out
- When the system **recovers from a crash**
 - Initialize **undo-list** and **redo-list** to empty
 - Scan the log backwards until a **<checkpoint L>** record is found:
 - if the record is **<T_i commit>**, add **T_i** to **redo-list**
 - if the record is **<T_i start>** and **T_i** is **not in redo-list**, add **T_i** to **undo-list**
 - for every **T_i** in **L**, if **T_i** is **not in redo-list**, add **T_i** to **undo-list**

Example of Recovery

- Go through the steps of the recovery algorithm on the following log

```

<T0 start>
<T0, A, 0, 10>
<T0 commit>
<T1 start>
<T1, B, 0, 10>
<T2 start>
<T2, C, 0, 10>
<T2, C, 10, 20>
<checkpoint {T1, T2}>
<T3 start>
<T3, A, 10, 20>
<T3, D, 0, 10>
<T3 commit>
  
```



Scan log backwards: **Undo T₁, T₂** in undo-list
 Scan log forwards: **Redo T₃** in redo-list

Recovery With Concurrent Transactions (Cont.)

- Recovery works as follows
 - Scan log **backwards** from the end of the log
 - During the scan, perform **undo** for each log record that belongs to a transaction in **undo-list**
 - Locate **the most recent <checkpoint L>** record

- Scan log **forwards** from the **<checkpoint L>** record till the end of the log
 - During the scan, perform **redo** for each log record that belongs to a transaction on **redo-list**

6.3.5 缓冲区管理

(日志缓冲区)

将一个块输出到稳定存储器的代价非常大，因此最好是一次输出多条日志记录。

为了达到这个目的，我们将日志记录写到主存的日志缓冲区中，

日志记录在输出至稳定存储器以前临时保存在那儿。

在日志缓冲区中可以集中多条日志记录，然后再用一次输出操作输出到稳定存储器中。

稳定存储器中的日志记录顺序必须与它们被写入日志缓冲区的顺序完全一致。

由于使用了日志缓冲区，日志记录在被输出到稳定存储器之前，

可能有相当长一段时间只存在于主存(易失性存储器)中。

由于系统发生崩溃时这种日志记录会丢失，所以我们需要遵守以下原则：

- ① 在日志记录 $\langle T \text{ commit} \rangle$ 输出到稳定存储器前，与事务 T 相关的所有日志记录必须已经输出到稳定存储器。
 - ② 在日志记录 $\langle T \text{ commit} \rangle$ 输出到稳定存储器后，事务 T 进入提交状态。
 - ③ 在主存中的数据块输出到数据库(非易失性存储器)前，与该块中数据有关的所有日志记录必须已经输出到稳定存储器。
- 这称为**先写日志** (Write-Ahead Logging, WAL) 规则。
- WAL 规则只要求日志中的 undo 信息已经被输出到了稳定存储器中，而 redo 信息允许以后再写。

Log Record Buffering

- **Log record buffering**
 - log records are **buffered in main memory**, instead of being output directly to stable storage
 - **Log records are output to stable storage when a block of log records in the buffer is full, or a log force operation is executed**
 - **Log force** is performed **to commit a transaction** by forcing all its log records (including the commit record) to stable storage
 - Several log records can be output using a single output operation, thus reducing the I/O cost
- **Write-ahead logging (WAL) rule for buffering log records**
 - Log records are output to stable storage in the order in which they are created
 - Transaction T_i enters the **commit state** only when the log record **$\langle T_i \text{ commit} \rangle$** has been output to stable storage
 - **Before** a block of data in main memory is output to the database, **all log records** pertaining to data in that block **must have been output to** stable storage

(数据库缓冲区)

人们可能期望事务在提交时会强制将修改过的所有块都输出到磁盘，这种策略称为强制策略。

另一种是非强制策略，即使一个事务修改过的某些块还没有写回到磁盘，也允许它提交。

非强制策略能使得事务更快地提交，而且它允许在一个块输出到稳定存储器之前可以将多次更新积聚起来，

这可以大大减少频繁更新的块的输出操作数量。

因此大多数系统所采用的标准方式是非强制策略。

人们可能期望被一个仍然活跃的事务修改过的块都不应该被写出到磁盘，这种策略被称作非抢占策略。

另一种是抢占策略，允许系统将修改过的块写到磁盘，即使做这些修改的事务还没有完全提交。

只要遵守先写日志规则，即使采用抢占策略，前文所述的所有恢复算法也都能正确工作。

而且非抢占策略不适合执行大量更新的事务，

因为缓冲区可能被已更新过但又不能被移出到磁盘的页面所占满，以致事务不能继续进行。

因此大多数系统所采用的标准方式是抢占策略。

Database Buffering

- Database maintains an **in-memory buffer** of data blocks
 - When a new block is needed, an existing block should be removed from buffer if the buffer is full
 - If the block chosen for removal has been updated, it must be output to disk
- **No update should be in progress on a block when it is output to disk, which is ensured as follows:**
 - Before writing a data item, transaction acquires **exclusive lock on block** containing the data item
 - Lock can be released once the write is completed.
 - Such locks held for short duration are called **latches** 门锁
 - Before a block is output to disk, the system acquires **an exclusive latch on the block**
 - Ensures no update can be in progress on the block

The End